TECHNICAL REFERENCE ARCHITECTURE

# Hush Music — Repository Architecture & File Reference Guide

A complete structural breakdown, UI customization map, and functional core reference for the Hush Music (BitChord client) Kotlin Multiplatform codebase.

| TARGET REPOSITORY | ARCHITECTURE | PLATFORMS | CORE ENGINE |
|---|---|---|---|
| `VantaKernal28/Hush-Music-` | `KMP + Compose Multiplatform` | `Android, TV, WearOS, JVM` | `ExoPlayer + FFmpeg JNI` |

### REPOSITORY SAFETY LEVEL LEGEND

**● SAFE TO EDIT (UI LAYER)**
Compose views, themes, color palettes, animations, and icons. Zero playback or backend crash risk.

**● EDIT WITH CAUTION**
ViewModels, state containers, navigation routes, settings binders, and build configurations.

**● DO NOT TOUCH (FUNCTIONAL CORE)**
ExoPlayer service, audio decoders, JNI C++, WebSocket sync, and stream scrapers. Fatal regression risk.

### Table of Contents


§ 1 Section 1: Complete File-by-File Breakdown (All Modules)

§ 1.1 Module `app` — Android Mobile Application

§ 1.2 Module `shared` & `sharedUi` — Domain & Compose UI

§ 1.3 Module `tv` & `wearos` — TV & Smartwatch Clients

§ 1.4 Module `native`, `backend` & `web` — DSP, Go Server & Web

§ 2 Section 2: UI Customization Map (Safe to Edit)

§ 3 Section 3: Functional Core (DO NOT TOUCH / Edit with Caution)

§ 4 Section 4: Safe UI Customization Workflow (7-Step Roadmap)


Here is the complete architectural inspection and file-by-file breakdown of the **Hush Music** (BitChord) repository ( `https://github.com/VantaKernal28/Hush-Music-/tree/main` ).

## 1. Complete File-by-File Breakdown

Hush Music is structured as a Kotlin Multiplatform (KMP) architecture targeting Android mobile, Android TV, Wear OS, and desktop JVM, supplemented by a high-performance C++ DSP native engine ( `native/` ), a Go WebSocket synchronization server ( `backend/` ), and a Web/Extension stack ( `web/` ).

## Root Project & Build System

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `settings.gradle.kts` | • EDIT WITH CAUTION | **Root Project Settings:** Declares project name ( `BitChord` ), repositories (Google, MavenCentral, JitPack), and conditionally includes `:app` , `:tv` , and `:wearos` when the Android SDK is detected, alongside the baseline `:shared` and `:sharedUi` KMP modules. |
| `build.gradle.kts` | • EDIT WITH CAUTION | **Root Build Script:** Configures plugin versions (Android Application/Library, Kotlin Multiplatform, Compose Multiplatform, KotlinX Serialization). |
| `gradle.properties` | • EDIT WITH CAUTION | **Gradle Tuning:** JVM memory constraints, AndroidX enablement, and KMP flags. |
| `keystore.properties.example` | • EDIT WITH CAUTION | **Signing Template:** Configuration template for APK signing keys. |
| `README.md` | • EDIT WITH CAUTION | **Project Documentation:** Feature catalog (lossless playback, crossfade, automix, dynamic artwork theming, lyrics providers). |

---

## Module: `app` (Android Application)

The main mobile application layer hosting Android OS integration, hardware bridges, services, and mobile-specific Compose screens.

### APP ENTRY, LIFECYCLE & HOST

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `app/src/main/AndroidManifest.xml` | • EDIT WITH CAUTION | **Android Manifest:** Declares permissions ( `FOREGROUND_SERVICE_MEDIA_PLAYBACK` , `WAKE_LOCK` , `MODIFY_AUDIO_SETTINGS` for USB DACs, `BLUETOOTH_CONNECT` , storage), deep links ( `music.youtube.com` , `youtu.be` , `bitchord://party` ), activities, services, and home-screen widgets. |
| `app/src/main/java/com/music/bitchord/BitChordApplication.kt` | • EDIT WITH CAUTION | **Application Entry:** Initializes Coil 3 image loader, registers custom audio components, and handles process-wide lifecycles. |
| `app/src/main/java/com/music/bitchord/MainActivity.kt` | • EDIT WITH CAUTION | **Main Activity & UI Root (6,278 lines):** Configures edge-to-edge window insets, instantiates `MainViewModel` , provides Compose composition locals, and implements `BitChordApp` (bottom bars, mini-player gestures, modal sheets, and root navigation). |
| `app/src/main/java/com/music/bitchord/AppIcon.kt` | • EDIT WITH CAUTION | **Dynamic App Icon Switcher:** Enables runtime switching between icon variants using `PackageManager` component-enablement flags. |
| `app/src/main/java/com/music/bitchord/ui/AndroidAppUiHost.kt` | • EDIT WITH CAUTION | **Android App UI Bridge:** Connects `sharedUi` 's `AppUiHost` contract to Android `AppSettings` and `Downloads` . |
| `app/src/main/java/com/music/bitchord/ui/MainViewModel.kt` | • EDIT WITH CAUTION | **Mobile Main ViewModel:** Manages search paging, home feed refreshing, account state, and library queries. |

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| app/src/main/java/com/music/bitchord/ui/navigation/AppNavigator.kt | EDIT WITH CAUTION | **Navigation Controller:** Backstack management for navigating between Albums, Artists, Playlists, Settings, and Detail pages. |

## UI THEMING, STYLING & ICONS

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| app/src/main/java/com/music/bitchord/ui/theme/Theme.kt | SAFE TO EDIT | **Core Mobile Theme:** Defines `DarkColors`, `LightColors`, `SFProDisplay` typography scale, `AccentRed`, and `SystemBarIcons` window inset contrast controller. |
| app/src/main/java/com/music/bitchord/ui/icons/SfIcons.kt | SAFE TO EDIT | **Apple SF Symbols:** Vector conversions for SF Symbols matching Apple Music's iconography. |
| app/src/main/java/com/music/bitchord/ui/icons/BrokenHeart.kt | SAFE TO EDIT | **Custom Icon:** Dislike / ignore action icon. |
| app/src/main/res/values/themes.xml | SAFE TO EDIT | **XML Base Theme:** Declares zero-actionbar edge-to-edge root theme so Compose controls all rendering. |
| app/src/main/res/font/ | SAFE TO EDIT | **Bundled Fonts:** Contains `sf_pro_display_regular.otf`, `sf_pro_display_medium.otf`, `sf_pro_display_semibold.otf`, `sf_pro_display_bold.otf`, and `sf_pro_display_heavy.otf`. |

## UI SCREENS & DETAIL SHEETS

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| app/src/main/java/com/music/bitchord/ui/screens/DetailScreen.kt | SAFE TO EDIT | **Detail Screen (Album/Artist/Playlist):** Renders header art, dynamic blurred hero background, track lists, and context actions. |
| app/src/main/java/com/music/bitchord/ui/screens/SettingsSheet.kt | SAFE TO EDIT | **Settings Sheet:** Full configuration drawer for audio quality ceilings, theme selections, font styles, and integrations. |
| app/src/main/java/com/music/bitchord/ui/screens/EqualizerScreen.kt | SAFE TO EDIT | **Equalizer Screen:** Custom multi-band EQ sliders, bass boost, and curve presets. |
| app/src/main/java/com/music/bitchord/ui/screens/HistoryScreen.kt | SAFE TO EDIT | **Listening History:** Chronological log of played songs with timestamp groupings. |
| app/src/main/java/com/music/bitchord/ui/screens/ListenTogetherScreen.kt | SAFE TO EDIT | **Listen Together UI:** Host/join shared Jam sessions, show connected peers and shared queue. |
| app/src/main/java/com/music/bitchord/ui/screens/SourcesScreen.kt | SAFE TO EDIT | **Audio Sources Manager:** Add, test, order, and toggle pluggable music sources (YouTube, JioSaavn, FLAC modules). |
| app/src/main/java/com/music/bitchord/ui/screens/DiscordScreen.kt | SAFE TO EDIT | **Discord RPC Config:** In-app Discord login and Rich Presence toggle. |
| app/src/main/java/com/music/bitchord/ui/screens/SpotifyLibraryScreen.kt | SAFE TO EDIT | **Spotify Importer UI:** View connected Spotify playlists and import them into local library. |

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `app/src/main/java/com/music/bitchord/ui/onboarding/OnboardingScreen.kt` | SAFE TO EDIT | **Onboarding Tour:** First-launch intro and permission requests. |
| `app/src/main/java/com/music/bitchord/ui/replay/ReplayScreen.kt` | SAFE TO EDIT | **Year in Review / Replay:** Story-based listening statistics, top songs, and artist breakdown. |
| `app/src/main/java/com/music/bitchord/ui/replay/ReplayPoster.kt` | SAFE TO EDIT | **Replay Share Card:** Generates shareable image posters of user stats. |

## UI COMPONENTS & VISUAL FX

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `app/src/main/java/com/music/bitchord/ui/components/MiniPlayer.kt` | SAFE TO EDIT | **Bottom Mini Player:** Floating pill showing active track, play/pause, swipe gestures, and artwork thumbnail. |
| `app/src/main/java/com/music/bitchord/ui/components/GlassNavBar.kt` | SAFE TO EDIT | **Frosted Navigation Bar:** Glassmorphic bottom navigation bar with responsive collapsing. |
| `app/src/main/java/com/music/bitchord/ui/components/FloatingBottomBar.kt` | SAFE TO EDIT | **Floating Bar Container:** Container coordinating mini player and tab bar elevation. |
| `app/src/main/java/com/music/bitchord/ui/components/LiquidGlass.kt` | SAFE TO EDIT | **Liquid Glass Effect:** AGSL shader modifier providing refraction and glass translucency. |
| `app/src/main/java/com/music/bitchord/ui/components/backdrop/` | SAFE TO EDIT | **Backdrop Shader Engine:** Real-time multi-layer blur, lens distortion, and shadow rendering engine ( `Shaders.kt` , `RuntimeShader.kt` , `LayerBackdrop.kt` ). |
| `app/src/main/java/com/music/bitchord/ui/components/FrostedTopBar.kt` | SAFE TO EDIT | **Translucent App Bar:** Blur-backed top app bar that dims dynamically on scroll. |
| `app/src/main/java/com/music/bitchord/ui/components/SongActionsMenu.kt` | SAFE TO EDIT | **Song Context Menu:** ⋯ menu with actions for Play Next, Add to Queue, Download, Share, Song Info. |

## PLAYER PLATFORM & MOTION

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `app/src/main/java/com/music/bitchord/ui/player/AndroidPlayerHost.kt` | EDIT WITH CAUTION | **Android Player Host:** Implements `sharedUi` 's `PlayerHost` interface on Android (canvas video decoding, volume hooks, audio outputs). |
| `app/src/main/java/com/music/bitchord/ui/player/CanvasArtworkPlayer.kt` | EDIT WITH CAUTION | **Canvas Video Player:** ExoPlayer-based looping video player for animated canvas backgrounds. |
| `app/src/main/java/com/music/bitchord/ui/player/MusicVideoPlayer.kt` | EDIT WITH CAUTION | **Music Video Renderer:** Displays YouTube music video streams synchronized with audio. |

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| app/src/main/java/com/music/bitchord/ui/player/PlayerSheetMotion.kt | EDIT WITH CAUTION | **Sheet Physics:** Physics-based drag and spring animation connecting the mini player to the full-screen player. |
| app/src/main/java/com/music/bitchord/ui/player/LyricsShareCard.kt | EDIT WITH CAUTION | **Lyrics Card Generator:** Formats selected lyrics lines into an Instagram/social media share card. |

## AUDIO PLAYBACK ENGINE (CRITICAL BACKEND)

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| app/src/main/java/com/music/bitchord/playback/PlaybackService.kt | DO NOT TOUCH | **Core Playback Engine (9,884 lines):** Foreground `MediaLibraryService` managing ExoPlayer, dual audio tracks for crossfading, notifications, Android Auto, lockscreen media controls, and stream routing. |
| app/src/main/java/com/music/bitchord/playback/PlayerConnection.kt | DO NOT TOUCH | **UI–Playback Bridge:** Manages `MediaController` connection, provides `playerState` to Compose, and exposes transport controls ( `play` , `pause` , `seek` , `skip` ). |
| app/src/main/java/com/music/bitchord/playback/CrossfadeController.kt | DO NOT TOUCH | **Gapless Crossfade:** Controls volume envelopes and dual-player synchronization for 0–12s crossfading. |
| app/src/main/java/com/music/bitchord/playback/AudioCache.kt | DO NOT TOUCH | **ExoPlayer Cache:** Disk cache for prefetching and offline caching of audio chunks. |
| app/src/main/java/com/music/bitchord/playback/audio/bitperfect/ | DO NOT TOUCH | **Bit-Perfect Engine:** Bypasses Android's audio mixer to deliver direct, bit-perfect PCM/DSD streams to external USB DACs. |
| app/src/main/java/com/music/bitchord/playback/audio/DspChain.kt | EDIT WITH CAUTION | **DSP Pipeline:** AudioProcessor chain hosting equalizer, peak limiter, and pitch/speed processors. |

## DATA & INTEGRATION

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| app/src/main/java/com/music/bitchord/data/settings/AppSettings.kt | EDIT WITH CAUTION | **App Preferences:** Datastore/StateFlow store for all settings (bitrates, themes, fonts, toggles). |
| app/src/main/java/com/music/bitchord/data/sources/SourceResolver.kt | EDIT WITH CAUTION | **Audio Source Resolver:** Resolves high-res FLAC from external modules or falls back to YouTube Music. |
| app/src/main/java/com/music/bitchord/download/DownloadService.kt | EDIT WITH CAUTION | **Download Manager Service:** Foreground service managing track and album downloads. |
| app/src/main/java/com/music/bitchord/download/MediaTagger.kt | EDIT WITH CAUTION | **Metadata Embedder:** Injects ID3, Vorbis, and MP4 metadata tags, covers, and synced lyrics into downloaded files. |

## Module: `shared` (KMP Domain & API Logic)

The cross-platform core containing data models, network clients, API scrapers, and DSP mathematical models.

### `SHARED/SRC/COMMONMAIN` (PURE COMMON LOGIC)

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| shared/src/commonMain/kotlin/com/music/bitchord/data/model/Models.kt | • EDIT WITH CAUTION | **Core Domain Models:** Declares `Song`, `Album`, `Artist`, `Playlist`, `MediaMetadata`, and `StreamUrl`. |
| shared/src/commonMain/kotlin/com/music/bitchord/data/listentogether/PartyModels.kt | • EDIT WITH CAUTION | **Listen Together Models:** Room state, member lists, and synchronization packets. |
| shared/src/commonMain/kotlin/com/music/bitchord/playback/EqualizerCurve.kt | • EDIT WITH CAUTION | **EQ Math:** Mathematical definitions of frequency response curves and presets. |
| shared/src/commonMain/kotlin/com/music/bitchord/playback/smart/TrackAnalysis.kt | • EDIT WITH CAUTION | **Automix Analysis Data:** Tempo, key, beats, and energy level models for DJ transitions. |

### `SHARED/SRC/JVMSHAREDMAIN` (APIS, NETWORK & SCRAPERS)

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/innertube/Innertube.kt | • DO NOT TOUCH | **YouTube Music API Engine:** Reverse-engineered YouTube Music client for search, browse, artist pages, and home feeds. |
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/innertube/potoken/PoTokenGenerator.kt | • DO NOT TOUCH | **Proof-of-Origin Token Engine:** Bypasses YouTube's bot-guard and playback integrity verification to obtain raw audio streams. |
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/innertube/StreamResolver.kt | • DO NOT TOUCH | **Stream URL Decryptor:** Resolves and decrypts Opus/AAC streaming URLs from YouTube backend responses. |
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/lyrics/LyricsRepository.kt | • EDIT WITH CAUTION | **Lyrics Orchestrator:** Unified manager aggregating lyrics from 10+ providers with fallback priorities. |
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/lyrics/ | • EDIT WITH CAUTION | **Lyrics Providers:** Individual scrapers/parsers for `LrcLib.kt`, `Musixmatch.kt`, `Genius.kt`, `KuGou.kt`, `Megalobiz.kt`, `SimpMusicLyrics.kt`, `PaxSenix.kt`, `BiniLyrics.kt`, `BetterLyrics.kt`, and `TtmlLyrics.kt`. |
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/spotify/SpotifyImporter.kt | • EDIT WITH CAUTION | **Spotify Library Sync:** Fetches Spotify playlists and maps metadata to YouTube Music catalogue. |
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/canvas/SpotifyCanvas.kt | • EDIT WITH CAUTION | **Spotify Canvas Fetcher:** Fetches Spotify vertical video loops for matching songs. |
| shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/canvas/AppleMusicCanvas.kt | • EDIT WITH CAUTION | **Apple Music Canvas Fetcher:** Fetches Apple Music animated album artwork. |

## Module: `sharedUi` (Compose Multiplatform Visual Layer)

The unified visual presentation layer shared across Android, Desktop, TV, and Tablet.

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/theme/ArtworkPalette.kt` | • SAFE TO EDIT | **Dynamic Color Extraction:** Extracts dominant colors from album art to dynamically tint background wash, accents, elevated surfaces, and dividers. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/icons/BitChordIcons.kt` | • SAFE TO EDIT | **Core Vector Icons:** Custom 2.2px rounded stroke icons for queue, lyrics, repeat, shuffle, and quality badges. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/NowPlayingScreen.kt` | • SAFE TO EDIT | **Full Player UI (4,249 lines):** The main player screen displaying album art, title, scrubber, playback controls, lyrics toggle, and queue drawer. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/PlayerLyrics.kt` | • SAFE TO EDIT | **Apple-style Synced Lyrics (3,283 lines):** Word-level and syllable-level animated lyric highlights with spring physics, blur depth, and auto-scrolling. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/MeshGradient.kt` | • SAFE TO EDIT | **Animated Mesh Gradient:** Fluid multi-color gradient mesh that continuously shifts based on the album art palette. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/PlayerControls.kt` | • SAFE TO EDIT | **Playback Transport Bar:** Play/pause, next, previous, shuffle, repeat, and time scrubbers. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/PlayerQueue.kt` | • SAFE TO EDIT | **Up Next Queue Drawer:** Reorderable drag-and-drop track queue with swipe-to-dismiss and history sections. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/HomeScreen.kt` | • SAFE TO EDIT | **Home Feed Screen:** Personalized carousels, Quick Picks, and Top Picks shelves. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/ExploreScreen.kt` | • SAFE TO EDIT | **Explore Screen:** Music charts, new releases, and mood/genre chips. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/LibraryScreen.kt` | • SAFE TO EDIT | **Library Screen:** User playlists, downloaded tracks, liked songs, and local device tracks. |
| `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/SearchScreen.kt` | • SAFE TO EDIT | **Search Screen:** Real-time query suggestions and search results grid. |

---

## Module: `tv` (Android TV Application)

10-foot UI designed for big screens with Android TV D-pad focus and navigation.

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `tv/src/main/java/com/music/bitchord/TvActivity.kt` | • EDIT WITH CAUTION | **TV Activity:** Main activity handling Android TV D-pad key events and lifecycle. |
| `tv/src/main/java/com/music/bitchord/ui/tv/player/TvNowPlayingScreen.kt` | • SAFE TO EDIT | **TV Now Playing Screen:** Large-format playback interface featuring lyrics on one half and giant cover art on the other. |

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `tv/src/main/java/com/music/bitchord/ui/tv/auth/TvAuthServer.kt` | EDIT WITH CAUTION | **QR Login Server:** Embedded HTTP server allowing users to log in on their TV by scanning a QR code with a phone. |

---

### Module: `wearos` (Wear OS Companion & Standalone App)

Wearable interface for remote control and on-wrist audio streaming.

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `wearos/src/main/java/com/music/bitchord/wear/WearActivity.kt` | EDIT WITH CAUTION | **Wear OS Activity:** Rotary input (crown) handling, round display scaling, and ambient mode. |
| `wearos/src/main/java/com/music/bitchord/wear/ui/NowPlayingScreen.kt` | SAFE TO EDIT | **Watch Player Screen:** Circular UI with compact transport buttons and volume arc. |
| `wearos/src/main/java/com/music/bitchord/wear/playback/WatchPlaybackService.kt` | DO NOT TOUCH | **Watch Playback Service:** Audio output directly to Bluetooth headphones paired to the smartwatch. |

---

### Module: `native` (C++ Audio DSP & JNI)

C++ performance core handling real-time audio analysis and specialized codecs.

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `native/analyzer/tempo_analysis.cpp` | DO NOT TOUCH | **BPM & Tempo Detector:** Autocorrelation-based beat detection for automix tempo matching. |
| `native/analyzer/mel_spectrogram.cpp` | DO NOT TOUCH | **Mel Spectrogram:** Computes frequency bins for audio waveform visualization and beat matching. |
| `native/loopmix/loop_mix_renderer.cpp` | DO NOT TOUCH | **DJ Loop Renderer:** Renders DJ-style seamless loop transitions across tracks. |
| `native/ffmpeg/ffmpeg_jni.cc` | DO NOT TOUCH | **FFmpeg JNI Bridge:** Connects Media3's `FfmpegAudioRenderer` to native FFmpeg libraries for decoding FLAC, ALAC, and DSD. |
| `native/third_party/signalsmith-stretch/` | DO NOT TOUCH | **Time/Pitch DSP Engine:** High-quality phase vocoder for real-time tempo shifting without altering pitch. |

---

### Module: `backend` (Go Server for "Listen Together")

Real-time WebSocket server coordinating group listening sessions.

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `backend/main.go` | EDIT WITH CAUTION | **Server Entrypoint:** HTTP server handling invite codes, health checks, and WebSocket upgrades. |
| `backend/hub/hub.go` | EDIT WITH CAUTION | **WebSocket Room Hub:** Coordinates message broadcast across connected party members. |

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `backend/party/party.go` | • EDIT WITH CAUTION | **Party State Machine:** Enforces host permissions, participant queues, and sync commands. |
| `backend/clock/clock.go` | • EDIT WITH CAUTION | **NTP-Style Time Synchronizer:** Syncs client clocks within milliseconds to ensure simultaneous playback across devices. |

### Module: `web` (Web Player & Chrome Extension)

| PATH | SAFETY LEVEL | ROLE & IMPORTANCE |
|---|---|---|
| `web/app/src/` | • EDIT WITH CAUTION | **Web Client:** React + TypeScript web app mirroring Hush Music features in a desktop browser. |
| `web/extension/` | • EDIT WITH CAUTION | **Browser Extension (WXT):** Companion browser extension providing YouTube and Spotify session credentials to bypass CORS and bot-guard protections. |

## 2. UI Customization Map (Safe to Edit)

When reskinning, restyling, or redesigning Hush Music, focus on the following files. Modifying these files will not introduce audio, database, or network regressions.

```
UI Reskinning Entry Points
 ├── Global Colors & Typography      ──> app/src/main/java/com/music/bitchord/ui/theme/Theme.kt
 ├── Dynamic Artwork Theming         ──> sharedUi/.../ui/theme/ArtworkPalette.kt
 ├── Status Bar Tinting              ──> sharedUi/.../ui/theme/StatusBarIcons.kt
 ├── Custom Stroke Icons             ──> sharedUi/.../ui/icons/BitChordIcons.kt
 ├── SF Symbols Icons                ──> app/src/main/java/com/music/bitchord/ui/icons/SfIcons.kt
 ├── Navigation & Glass Bar          ──> app/src/main/java/com/music/bitchord/ui/components/GlassNavBar.kt
 ├── Mini Player Bar                 ──> app/src/main/java/com/music/bitchord/ui/components/MiniPlayer.kt
 ├── Now Playing Screen              ──> sharedUi/.../ui/player/NowPlayingScreen.kt
 ├── Mesh Gradient Animations        ──> sharedUi/.../ui/player/MeshGradient.kt
 ├── Apple Synced Lyrics View        ──> sharedUi/.../ui/player/PlayerLyrics.kt
 ├── Transport & Sliders             ──> sharedUi/.../ui/player/PlayerControls.kt & ThinSlider.kt
 ├── Up Next Queue Drawer            ──> sharedUi/.../ui/player/PlayerQueue.kt
 ├── Home / Explore / Library Feeds  ──> sharedUi/.../ui/screens/
 └── Detail Screens (Albums/Artists) ──> app/src/main/java/com/music/bitchord/ui/screens/DetailScreen.kt
```

### 1. Color Palette, Theming & Typography

- `app/src/main/java/com/music/bitchord/ui/theme/Theme.kt`:
- **`DarkColors`** / **`LightColors`**: Material 3 color schemes (`primary`, `surface`, `background`, `surfaceVariant`, `outline`). Edit these to change global app backgrounds and text contrast.
- **`AccentRed`**: Highlight color used on badges and rank indicators (`#FA2D48`).
- **`BitChordTypography`**: Text scale for all headlines, titles, body sizes, and font weights.
- **`SFProDisplay`**: Font family mapping; change this to bundle and apply your own typography.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/theme/ArtworkPalette.kt`:
- Controls **dynamic palette extraction** from album artwork (`background`, `wash`, `elevated`, `accent`, `divider`). Tweak math here to make player backgrounds darker, more saturated, or matte.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/theme/StatusBarIcons.kt`:

- Controls luminance calculations determining whether the OS status and navigation bar icons are rendered dark or light.

### 2. Iconography & Visual Assets

- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/icons/BitChordIcons.kt` :
- Contains vector path definitions for primary UI controls: `Queue` , `LyricsQuote` , `Repeat` , `Shuffle` , `VolumeHigh` , `Equalizer` , etc.
- `app/src/main/java/com/music/bitchord/ui/icons/SfIcons.kt` :
- Mapping for Apple SF Symbols used throughout detail sheets and popup menus.
- `sharedUi/src/commonMain/composeResources/drawable/` :
- Play, pause, next, previous vector XML assets.

### 3. Now Playing & Full Player Layer

- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/NowPlayingScreen.kt` :
- Controls the full-screen playback layout: album cover aspect ratio, artwork corner radius, title/artist typography, margins, and transition layout between portrait and landscape.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/MeshGradient.kt` :
- Canvas shader for the animated multi-color liquid backdrop. Adjust blur radius, color stop offsets, and animation speed.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/PlayerLyrics.kt` :
- Apple Music-style lyrics screen: typography, word glow, spring scroll curves, blur opacity on inactive lines, and karaoke highlights.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/PlayerControls.kt` :
- Sizing, alignment, and spacing of Play/Pause, Skip, Repeat, Shuffle, and Audio Output picker buttons.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/ThinSlider.kt` :
- Progress bar / scrubber: track height, thumb size, expansion animation when dragged, and buffer indicator.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/PlayerQueue.kt` :
- Up Next queue sheet: row card shapes, drag handles, swipe dismiss colors, and queue section headers.

### 4. Navigation & Surrounding Shell

- `app/src/main/java/com/music/bitchord/ui/components/MiniPlayer.kt` :
- Mini player pill: height, background blur, rounded corners, progress hairline, and swipe animation.
- `app/src/main/java/com/music/bitchord/ui/components/GlassNavBar.kt` :
- Bottom navigation bar: glass pill radius, icon tinting, active tab indicators, and collapse on scroll.
- `app/src/main/java/com/music/bitchord/ui/components/LiquidGlass.kt` :
- AGSL shader implementation of frosted glass refraction.
- `app/src/main/java/com/music/bitchord/ui/components/FrostedTopBar.kt` :
- Top header styling and blur scrims.

### 5. Content Feeds & Pages

- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/HomeScreen.kt` : Home feed layout, card sizes, and horizontal carousel margins.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/ExploreScreen.kt` : Mood & genre grid layouts and charts.
- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/LibraryScreen.kt` : Playlist and offline downloads list item layouts.

- `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/screens/SearchScreen.kt` : Search bar design, auto-complete item styling, and result tiles.
- `app/src/main/java/com/music/bitchord/ui/screens/DetailScreen.kt` : Album / Artist / Playlist detail pages (large header artwork, action buttons, track rows).

---

# 3. Functional Core (DO NOT TOUCH / Edit with Caution)

These backend and engine files contain delicate logic. Modifying them without exact knowledge will break audio decoding, background playback, network token verification, or cause app crashes.

### Critical Files & Impact Analysis

#### 1. AUDIO PLAYBACK & SERVICE LIFECYCLE

- `app/src/main/java/com/music/bitchord/playback/PlaybackService.kt`
- **Role:** Manages the Android `MediaSession` / `MediaLibraryService`, dual-ExoPlayer instances for crossfading, system notifications, Android Auto, and background wake locks.
- **What Breaks If Modified:** The app will be killed by Android OS in the background; media notification controls will disappear or freeze; Bluetooth/headset button controls will stop responding; Android Auto will fail to initialize; audio playback will glitch on track change.
- `app/src/main/java/com/music/bitchord/playback/PlayerConnection.kt`
- **Role:** The IPC bridge connecting Compose UI to `PlaybackService` via `MediaController`.
- **What Breaks If Modified:** UI will fall out of sync with actual audio (progress bar won't move, play/pause buttons will reflect the wrong state, queue reorders won't update the underlying playlist).
- `app/src/main/java/com/music/bitchord/playback/CrossfadeController.kt`
- **Role:** Enforces volume curves, timing, and pre-buffering across the primary and secondary ExoPlayer instances.
- **What Breaks If Modified:** Causes sudden audio volume jumps, audio clipping, or silence during track transitions.
- `app/src/main/java/com/music/bitchord/playback/audio/bitperfect/BitPerfectAudioSink.kt`
- **Role:** Custom ExoPlayer `AudioSink` communicating directly with USB host hardware for bit-perfect output.
- **What Breaks If Modified:** High-resolution FLAC/DSD audio over USB DACs will produce loud digital distortion, hiss, or fatal crashes.

#### 2. STREAMING & YOUTUBE MUSIC API

- `shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/innertube/Innertube.kt`
- **Role:** Reverse-engineered YouTube Music client that queries browse endpoints, search suggestions, and stream payloads.
- **What Breaks If Modified:** Search, home feed, recommendations, and album browsing will fail with HTTP 400/403 or JSON parsing errors.
- `shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/innertube/potoken/PoTokenGenerator.kt`
- **Role:** Generates YouTube Proof-of-Origin (PoToken) security hashes via hidden JavaScript execution.
- **What Breaks If Modified:** YouTube will detect the app as an unauthorized bot and block all audio streams (playback will halt with `HTTP 403 Forbidden`).
- `shared/src/jvmSharedMain/kotlin/com/music/bitchord/data/innertube/StreamResolver.kt`
- **Role:** Parses audio stream manifests, decodes cipher signatures, and selects optimal bitrates.
- **What Breaks If Modified:** ExoPlayer will receive invalid stream URLs and throw `UnrecognizedInputFormatException` or `BehindLiveWindowException`.

#### 3. PLUGGABLE SOURCES & MODULE ENGINE

- `app/src/main/java/com/music/bitchord/data/sources/SourceResolver.kt`

- **Role:** Orchestrates multi-source resolution (matching YouTube metadata against FLAC/ALAC sources).
- **What Breaks If Modified:** Lossless playback fallback logic breaks, resulting in songs failing to play or mismatched audio versions.
- `app/src/main/java/com/music/bitchord/data/sources/module/QuickJsExecutor.kt`
- **Role:** Embedded JavaScript runtime executing third-party external source modules.
- **What Breaks If Modified:** Third-party audio sources and scrapers will fail to execute.

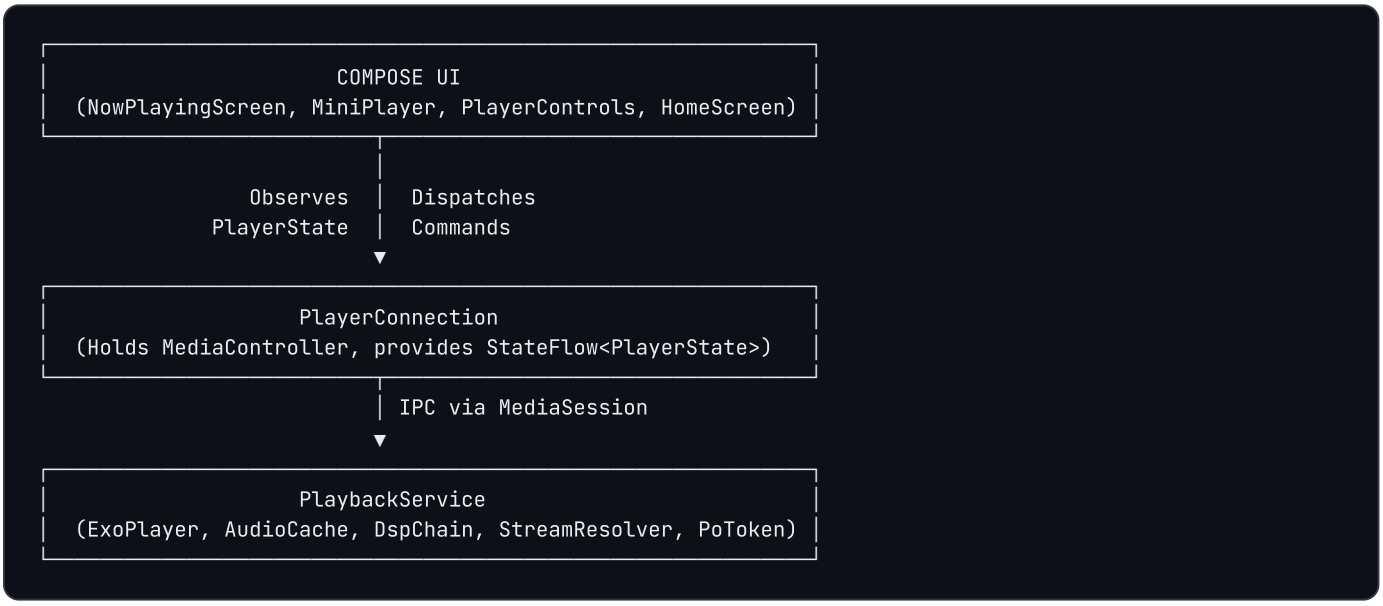
#### 4. CACHE, DATABASE & SETTINGS STORAGE

- `app/src/main/java/com/music/bitchord/data/settings/AppSettings.kt`
- **Role:** Manages all persistent preferences using KotlinX Serialization and DataStore/SharedPreferences.
- **What Breaks If Modified:** Modifying key names or data types without proper migrations will corrupt user settings and cause `SerializationException` crashes at startup.
- `app/src/main/java/com/music/bitchord/playback/AudioCache.kt`
- **Role:** Manages the LRU disk cache for streamed audio blocks.
- **What Breaks If Modified:** Excessive disk usage, out-of-memory errors, or corrupted audio playback when offline.

---

### How the UI Connects to the Core (Contract Architecture)

The architecture uses a clean separation of concerns via reactive state flows:




1. **State Observation Only:** The UI never accesses ExoPlayer directly. It strictly observes `PlayerConnection.playerState` (which provides `song`, `isPlaying`, `durationMs`, and `position: PlaybackPosition`).
2. **Action Dispatch Only:** User interactions invoke high-level callbacks on `PlayerConnection` (`play()`, `pause()`, `seekTo(ms)`, `skipNext()`, `skipPrevious()`, `selectSource()`).
3. **Safe Frontend Rule:** As long as custom UI code only reads from `playerState` and invokes methods on `playerConnection` or `MainViewModel`, you can completely replace layouts, colors, animations, and shapes without affecting the audio engine.

---

## 4. Safe UI Customization Workflow

Follow this step-by-step checklist to customize or reskin Hush Music cleanly without causing compile errors or audio regressions:

### Step 1: Branding, App Name & Static Assets

1. **App Identity:**
2. Update string resources in `app/src/main/res/values/strings.xml` ( `app_name` ).
3. **Launcher Icons & Logos:**
4. Replace vector drawables and mipmaps in `app/src/main/res/drawable/` ( `ic_logo.xml` , `ic_launcher_foreground.xml` ) and `sharedUi/src/commonMain/composeResources/drawable/ic_logo.xml` .

### Step 2: Global Theme & Typography

1. **Typography & Fonts:**
2. Place your custom `.ttf` / `.otf` fonts into `app/src/main/res/font/` .
3. Open `app/src/main/java/com/music/bitchord/ui/theme/Theme.kt` and update `SFProDisplay` to point to your new font resources.
4. Adjust `BitChordTypography` size, weight, and letter spacing tokens.
5. **Colors & Dark/Light Themes:**
6. In `Theme.kt` , modify `DarkColors` and `LightColors` color definitions ( `primary` , `surface` , `background` , `surfaceVariant` , `outline` ).
7. Change `AccentRed` if you want a different signature highlight color.

### Step 3: Dynamic Palette & Backdrops

1. **Artwork Tint Behavior:**
2. Open `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/theme/ArtworkPalette.kt` .
3. Adjust luminance thresholds and saturation multipliers if you prefer muted, darker, or more vibrant tints derived from album covers.
4. **Fluid Mesh Gradients:**
5. In `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/MeshGradient.kt` , tweak `FallbackColors` , blur radii, and animation speed for the dynamic now-playing background.

### Step 4: Shell, Navigation & Mini-Player

1. **Navigation Bar:**
2. Edit `app/src/main/java/com/music/bitchord/ui/components/GlassNavBar.kt` to modify the bottom navigation pill shape, padding, elevation, and selected tab pill animation.
3. **Mini Player:**
4. Edit `app/src/main/java/com/music/bitchord/ui/components/MiniPlayer.kt` to adjust mini-player height, artwork border radius, and play/pause button sizing.

### Step 5: Now Playing Player Screen & Lyrics

1. **Player Layout:**
2. Edit `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/NowPlayingScreen.kt` to rearrange playback controls, artwork margins, and sheet headers.
3. **Scrubber / Sliders:**
4. Customize `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/ThinSlider.kt` for seek bar height and thumb dynamics.
5. **Lyrics Visuals:**
6. Edit `sharedUi/src/jvmSharedMain/kotlin/com/music/bitchord/ui/player/PlayerLyrics.kt` to adjust lyric text size, active word glow, and blur amount on inactive lines.

### Step 6: Content Screens & Cards

1. **Home, Explore, Library & Detail:**
2. Customize card sizes, grid spans, and spacing in:
   - `HomeScreen.kt`
   - `ExploreScreen.kt`
   - `LibraryScreen.kt`
   - `DetailScreen.kt`

### Step 7: Verification

1. Run `./gradlew compileDebugKotlin` or assemble the debug APK to verify zero syntax/type regressions.
2. Test playback with screen locked to ensure audio continues playing seamlessly through the background service.